

Anthropic-Lecture-Prinzipien im JANS AI Hub

Datum: 29.07.2026

Anlass: Anthropic Claude-Code-Lecture, 32 Slides (fotografiert von Raphael Jans,

Quelle OneDrive/AD - 01 Geschaeftsfuehrung/JANS AI/260729 Antrophic Lecture/)

Auftrag: «Implementiere diese Grundlagen in die Architektur von JANS AI Hub.»

Destillat der Lecture: wissen/claude-code/wiki/lecture-260729-anthropic.md

Ausgangslage: gemessen, nicht geschaezt

Vor der Umsetzung wurde der Hub gegen jedes Lecture-Prinzip gemessen. Der Hub erfuehlt die meisten davon bereits ueber dem in der Lecture gezeigten Niveau: 53 Skills, 44 Sub-Agenten, 19 importierte Rules, 16 Connectoren, Hooks (SessionStart, UserPromptSubmit, PreToolUse), Scheduled Tasks, 19 bereichsweise CLAUDE.md unter wissen/.

Die Luecken lagen darum nicht bei «mehr davon», sondern an vier Stellen, an denen die Lecture einen anderen Weg zeigt als den bisher gewaehlten, plus einer Stelle, an der die Lecture selbst zu kurz greift.

Lecture-Prinzip	Ist-Zustand 29.07.2026 (gemessen)	Befund
Tip #5/#6 Kontext tunen	Grundkontext 105'573 B (~26'400 Token) in JEDER Session, davon 34 % Betriebsprotokoll	Luecke
Kontext-Hierarchie (Global-Ebene)	~/.claude/CLAUDE.md existierte nicht — jede Session ausserhalb des Repos ohne JANS-Kontext	Luecke
Tip #7 Konfiguration einchecken	.mcp.json gitignored, obwohl secret-frei; Connectoren pro Station von Hand	Luecke
SDK als Unix-Werkzeug	5 Scripts mit claude -p, alle --output-format text — keine maschinell auswertbaren Ergebnisse	Luecke
Tip #3 Werkzeuge beibringen	16 Connectoren, aber nur 5 im README, kein zentraler Index	Teil-Luecke
Multi-Claude	0 Worktrees, sequenzieller Betrieb	bewusst offen (Speicher)
Sub-CLAUDE.md on demand	19 Stueck unter wissen/	bereits erfuehlt
Werkzeuge/Sub-Agenten/Hooks	weit ueber Lecture-Niveau	bereits erfuehlt

Die sechs Massnahmen

1. Kontext-Diaet 2.0 (Tip #5 + #6)

Die Lecture stellt zwei scheinbar widerspruechliche Tips nebeneinander: «mehr Kontext = kluegeres Claude» und «nimm Dir Zeit, den Kontext zu tunen — automatisch oder lazily?».

Aufloesung: mehr **relevanter** Kontext hilft, mehr **irrelevanter** verdraengt ihn.

rules/auto-verbesserungen.md war auf 36'029 B gewachsen (34 % des gesamten Grundkontexts) und bestand ueberwiegend aus Betriebsprotokoll — launchd-Inventare, vm_stat-Messwerte, Vorfallschroniken. Bei einer Offerte, einer Baurechtsfrage oder einer Mail spielt davon nichts eine Rolle.

Umgesetzt: Trennung in

- rules/auto-verbesserungen.md — behaelt einen kompakten Abschnitt «Betrieb» mit den **Kurzregeln** (was zu tun ist) und alle Verhaltensregeln. Weiterhin importiert.
- rules/betrieb-chronik.md — **nicht importiert**. Enthaelte die vollstaendigen Belege, Messwerte und Vorfallsanalysen sowie einen Kopf, der benennt, wer sie liest (vollgas-chef-radar, hub-chef, jede Session an scripts/, launchd oder Takten).

Wirkung: Grundkontext von 105'573 B auf 87'398 B, minus 17 %. Jede Betriebsregel bleibt als Kurzregel greifbar (gegengeprueft ueber die Stichworte lauf-gate, vm_stat, EXCLUDE_RE, Liefer-Delta, mount volume, Zweitinstanz).

Verankert als Nachtrag 29.07. im Eintrag «260719 — Kontext-Diaet»: vor jedem neuen @-Import und vor jedem Anwachsen einer importierten Rule wird entschieden, ob sie in nahezu jede Session gehoert. Belege und Chroniken sind nie Grundkontext.

2. User-Level `~/claude/CLAUDE.md` (Kontext-Hierarchie)

Die Lecture-Matrix kennt vier Memory-Ebenen. Der Hub nutzte nur die Projekt-Ebene. Folge: sobald Claude Code ausserhalb von ~/Developer/jans-ai-hub startete — im Projektordner, im Downloads-Ordner, per mini — fehlten DNA, Anrede-Regeln, Umlaut-Konvention und Absenderadresse vollstaendig.

Umgesetzt: templates/user-level/CLAUDE.md (3'269 B) als kanonische Quelle auf dem NAS, verteilt via scripts/user-claude-sync.sh (--check prueft, --alle zieht den Mac Mini mit). Inhalt bewusst minimal: Sprache, Absender, Anrede-Default, Ablage-Konvention, Quellenpflicht, Layout-Eckwerte, Grenzen — plus der Wegweiser zum Hub.

Bewusst kein Symlink aufs NAS: Der Zweck dieser Datei ist gerade, zu greifen, wenn der Hub nicht erreichbar ist. Ein toter Symlink bei fehlendem Mount wuerde sie aushebeln. Drift wird stattdessen per md5 gemessen (--check).

Verifiziert: beide Stationen identisch, --check beidseitig «OK».

3. SDK mit JSON-Ausgabe (Slide «Claude Code SDK»)

Die Lecture zeigt den SDK als Unix-Werkzeug: `pipe in, pipe out, --output-format json`, Weiterverarbeitung mit `jq`. Im Hub liefen alle funf automatischen Aufrufe mit

`--output-format text` — kein Lauf gab je maschinell auswertbare Daten zurueck.

Das ist genau die Luecke, die Rule 260729 beschreibt: der Leerlauf-Waechter muss den

Liefer-Delta messen, hatte aber nur `lastRunAt` (markiert den Start, nicht die Lieferung) und freien Text.

Umgesetzt: `scripts/claude-run.sh` als gemeinsamer Wrapper. Er ruft `claude -p` mit

`--output-format json`, schreibt eine Journalzeile nach

`logbuch/laeufe/YYYYMMDD-laeufe.jsonl (cost_usd, duration_ms, num_turns, is_error, result_len, session_id, result_tail)` und gibt auf `stdout` **nur den Ergebnistext** aus.

Warum ein Wrapper und keine Flag-Aenderung in den Loops: Der `vollgas-runner` enthaelt eine sorgfaeltig kalibrierte Blindgaenger-Erkennung, die auf Textlaenge und Formulierung der Antwort prueft (`${#OUT}` unter 400 Zeichen). Rohes JSON haette sie stillschweigend ausser Kraft gesetzt — der Runner haette Blindgaenger nicht mehr erkannt. Der Wrapper liefert weiterhin reinen Text, die Erkennung bleibt unveraendert wirksam.

Im Test aufgedeckt und behoben: `claude` schreibt Warnungen auf `stderr`; mit `2>&1`

landeten sie vor dem JSON und liessen das Parsing scheitern. `stdout` und `stderr` werden darum getrennt erfasst.

Umgestellt wurden alle vier Aufrufstellen: `vollgas-runner.sh` (Hauptlauf und Blindgaenger-Retry), `wissens-trigger.sh` und `dispatch-run.sh`. Nach der Umstellung existiert im gesamten `scripts/-`Bestand kein `--output-format text` mehr.

Das Journal ist bewusst NICHT versioniert (`logbuch/laeufe/` in `.gitignore`, analog zu `dispatch/`): es enthaelt je Lauf einen Antwort-Ausschnitt, der bei operativen Laeufen (Mahnwesen, Zahlungen, Mail) Kundendaten und Betraege tragen kann — das gehoert nicht ins GitHub-Backup. Der Leerlauf-Waechter liest die lokale Datei seiner eigenen Station; die Loops sind ohnehin stationsgebunden verteilt.

4. `.mcp.json` einchecken + Werkzeug-Index (Tip #3 + #7)

`.mcp.json` stand in `.gitignore` unter «Secrets — NIEMALS committen». Die Pruefung des Inhalts zeigt: die Datei enthaelt **keine Credentials**, nur zwei Server-Namen, einen

Script-Pfad und eine HTTPS-URL. Die Geheimnisse liegen im Zertifikat

`~/ .cli-m365-cert-combined.pem` und in den `*.env` — beide bleiben ignoriert.

Umgesetzt: `.mcp.json` aus `.gitignore` genommen (mit begründendem Kommentar an Ort und Stelle), kanonische Fassung aufs NAS. Damit bekommt jede Station und jeder künftige Mitarbeitende die MCP-Server automatisch statt von Hand.

Werkzeug-Index: `connectors/README.md` dokumentierte 5 von 16 Connectoren. Neu trägt er eine Kopftabelle mit **allen 16** — Zweck, Einstiegs-Flags (aus den Scripts ausgelesen, nicht geraten) und Zugangsweg —, inklusive derer, die bei einem Skill liegen. `CLAUDE.md` verweist neu im Abschnitt «Werkzeuge / Connectoren» darauf, mit der Konvention, einen Connector zuerst per `--hilfe` selbst zu befragen (Lecture: «Use `-h` to check how to use it»).

5. Multi-Claude unter Speicher-Deckel (Slide «Interlude: Multi-claude»)

Die Lecture empfiehlt Parallelität über Worktrees, `tmux` und parallele Jobs. Der Hub hat am 28.07.2026 aus gutem Grund das Gegenteil eingezogen: einen stationsweiten Prozess-Deckel (`lauf-gate.sh`), nachdem gleichzeitige Läufe die Maschine in den Swap getrieben hatten.

Beides ist richtig. **Entscheid Raphaels:** Multi-Claude umsetzen, aber im Rahmen des Arbeitsspeichers der jeweiligen Station.

Umgesetzt: `scripts/multi-claude.sh`

- berechnet die Instanzzahl aus dem **real verfügbaren** Speicher
(`vm_stat free+inactive+purgeable`, Druckkriterium `kern.memorystatus_vm_pressure_level`)
— dieselbe Metrik wie das Gate, eine Messung für zwei Fragen
- `slots = min( (frei - Reserve) / Bedarf_je_Instance , Stationsmaximum - bereits_aktive )`
- Reserve und Maximum je Station identisch zum Gate (MacBook 2 / 3 GB, Mini 3 / 4 GB)
- `--slots` kann den berechneten Deckel nur **senken**, nie anheben
- prüft den Speicher **erneut**, bevor ein nachrückender Auftrag startet, und stellt bei Knappheit zurück statt zu starten
- jeder Auftrag in einem eigenen git worktree auf der **SSD**
(`~/Developer/jans-worktrees/`), nie über den SMB-Mount (Rule 260726)
- Worktrees werden nach dem Lauf nur entfernt, wenn sie **keine** Änderungen enthalten — sonst bleiben sie stehen und die Arbeit ist nicht verloren
- läuft über `claude-run.sh`, jeder Lauf landet im Journal
- `--trocken` plant ohne zu starten

Abgrenzung zum Gate: Das Gate beantwortet «darf DIESER Lauf starten?» (ja/nein), das Script «wie viele Laeufe traegt die Station jetzt?» (eine Zahl).

Beide Pfade nachgemessen (Lehre 28.07.: eine Schutzmechanik ist erst fertig, wenn Abweisungs- UND Freigabepfad je einmal belegt sind):

- **Abweisung** — MacBook Pro, 3'820 MB frei bei Druck 2: «ABBRUCH: Speicherdruck 2 — die Station swappt bereits.» Ursache des Drucks gemessen: zwei `tapir-archicad-MCP`-Prozesse mit zusammen 2.8 GB (bereits als offener Punkt im Radar gefuehrt, nicht angetastet).
- **Freigabe** — Mac Mini, 13'235 MB frei bei Druck 1, 0 aktive Laeufe: «Slots: 3 parallel (Speicher traegt 3, Prozessdeckel erlaubt 3)» bei 4 Auftraegen. Rechnung stimmt: $(13'235 - 4'000) / 3'000 = 3$, Deckel $3 - 0 = 3$.
- **Worktree-Mechanik** separat geprueft: anlegen aus dem SSD-Klon, Status-Pruefung, Entfernen — alles sauber.

`MAX_LAEUFE`, `MIN_FREI_MB` und `PRO_INSTANZ_MB` sind wie im Gate per Umgebungsvariable ueberschreibbar. Das **Druck-Kriterium bewusst nicht** — eine swappende Maschine soll sich nicht per Umgebungsvariable gesundschreiben lassen.

6. Wissen ablegen

Neue KB `wissen/claude-code/` — Wissen ueber das Werkzeug, auf dem der Hub ruht: `raw/` mit den 32 Slides, `wiki/lecture-260729-anthropic.md` (vollstaendiges Destillat), `wiki/kontext-architektur.md` (gemessener Ist-Zustand der vier Schichten samt Budget), `INDEX.md`, `QUESTIONS.md` (fuenf offene Punkte), `CHANGELOG.md`.

Was bewusst NICHT umgesetzt wurde

- **Enterprise-Policy-Ebene** (`/Library/Application Support/ClaudeCode/`): fuer zwei Stationen Overhead. Wird relevant, sobald weitere Mitarbeitende eigene Stationen bekommen. Als offener Punkt erfasst.
- ``/install-github-app`` (@claude auf Issues/PRs): der Hub nutzt GitHub nur als Backup, der Handy-Weg ueber `dispatch-run.sh` ist etabliert. Als offener Punkt erfasst.
- **User-Level-Slash-Commands** (`~/ .claude/commands/`): waere fuer `/nas`, `/status`, `/m365` sinnvoll, aber nicht Teil des Auftrags. Als offener Punkt erfasst.

Nebenfunde aus der Messung

1. **Workspace nicht «trusted»**. Headless-Laeufe aus `/Volumes/daten/jans-ai-hub` melden

«Ignoring 29 permissions.allow entries from .claude/settings.json: this workspace has not been trusted». Die 29 Berechtigungen der geteilten `settings.json` greifen dort also **nicht**. Zu klären, ob das die produktiven Loops betrifft — siehe offene Punkte.

2. **Cache-Creation dominiert die Kosten kleiner Laeufe.** Ein Testlauf mit dem Prompt «Antworte mit genau dem Wort: OK» erzeugte 89'618 Cache-Creation-Token und 0.54 USD — bei 2 Input- und 4 Output-Token. Der zweite Lauf: 0.42 USD, 470 Sekunden, ein Turn, zwei Zeichen Antwort. Der Grundkontext ist der Kostentreiber, nicht die Aufgabe. Das ist das stärkste Argument fuer die Kontext-Diaet und dafuer, wenige lange statt vieler kurzer automatischer Laeufe zu fahren.
3. **Startzeit rund 7-8 Minuten je headless Lauf** (MCP-Server-Start). Bei Loops faellt das nicht auf, macht aber jeden trivialen automatischen Lauf unwirtschaftlich.
4. **Auch lesende git-Befehle haengen ueber SMB.** `git status --porcelain` gegen das NAS-Repo lief in den 2-Minuten-Timeout, waehrend im Hintergrund ein Lauf und der 15-Min-Committer aktiv waren. Rule 260726 nennt nur schreibende Befehle — praezisiert in `rules/betrieb-chronik.md`, Eintrag 260729.
5. **Zwei verwaiste Claude-Worktrees** unter `.claude/worktrees/` (amazing-feistel, jolly-brattain) auf einem alten Commit. Nicht angetastet, aber aufräumenswert.

Geaenderte und neue Dateien

Neu

- `rules/betrieb-chronik.md`
- `templates/user-level/CLAUDE.md`
- `scripts/user-claude-sync.sh`
- `scripts/claude-run.sh`
- `scripts/multi-claude.sh`
- `.mcp.json` (auf dem NAS, neu versioniert)
- `wissen/claude-code/` (KB mit 32 Rohbildern und 4 Wiki-Dateien)
- dieses Konzept

Geaendert

- `rules/auto-verbesserungen.md` (36'029 → rund 17'000 B, Abschnitt «Betrieb», Nachtrag 260719)
- `CLAUDE.md` (Abschnitt «Werkzeuge / Connectoren», User-Level-Zeile, `.mcp.json`-Status)
- `connectors/README.md` (Werkzeug-Index aller 16 Connectoren)
- `.gitignore` (`.mcp.json` freigegeben, mit Begründung)